

Architecture et Programmation Web

Séance 3 : Introduction à Jinja2

Nicolas Burny

13 mars 2026

ECAM

Introduction

Qu'est-ce que Jinja ?

- Un **moteur de templates** pour le langage Python.
- Permet de générer dynamiquement des documents (HTML, XML, CSV, etc.).
- Conçu pour être rapide, sécurisé et facile à utiliser.
- Très largement utilisé avec le framework **Flask**.
- Syntaxe inspirée du système de templates de Django.

Qu'est-ce qu'un moteur de templates ?

- Un logiciel conçu pour combiner des **données** avec un **modèle de texte** (template).
- **Le Template** : Un fichier texte (souvent HTML) contenant des balises spéciales (*placeholders*).
- **Le Rendu** : Le moteur remplace les balises par les valeurs réelles et exécute la logique (boucles, conditions).
- **Résultat** : Un document final (HTML) envoyé au navigateur de l'utilisateur.

Pourquoi utiliser un moteur de templates ?

- **Séparation des préoccupations (SoC)** : Isoler la **logique métier** (Python) de la **couche de présentation** (HTML).
- **Maintenabilité** : Éviter la manipulation fastidieuse de chaînes de caractères HTML dans le code Python.
- **Réutilisation (DRY)** : Utiliser l'héritage de templates pour ne pas dupliquer les éléments communs (headers, menus).
- **Collaboration** : Permettre aux designers de modifier le visuel sans risquer de casser la logique applicative.

Exemple de template Jinja

```
<!DOCTYPE html>
<html>
<head>
  <title>{{ title }}</title>
</head>
<body>
  <h1>Bonjour {{ user.name }} !</h1>
  <ul>
    {% for item in items %}
      <li>{{ item }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

Utilisation avec Flask

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def index():
    # Flask cherche automatiquement dans le dossier 'templates/'
    return render_template(
        'index.html',
        title="Ma Page",
        user={"name": "Alice"},
        items=["Pomme", "Banane", "Cerise"]
    )
```

Syntaxe Fondamentale

`{{ ... }}` **Expression** : Affiche le résultat (variable, opération).

`{% ... %}` **Instruction** : Contrôle du flux (loops, conditions, extends).

`{# ... #}` **Commentaire** : Ignoré au rendu.

Boucles et Conditions

Données

```
users = [  
  {"name": "Alice",  
   "is_active": True},  
  {"name": "Bob",  
   "is_active": True},  
  {"name": "Charlie",  
   "is_active": False}  
]
```

Template

```
{% for user in users %}  
  {% if user.is_active %}  
    <li>{{ user.name }}</li>  
  {% endif %}  
{% endfor %}
```

Rendu

```
<li>Alice</li>  
<li>Bob</li>
```

Filtres (Filters)

Le concept de Filtres

Un **filtre** permet de transformer une donnée avant de l'afficher. On utilise le symbole "pipe" (|) pour l'appliquer à une variable.

Syntaxe :

```
{{ variable | nom_du_filtre }}
```

Exemple concret :

```
<!-- Si la variable 'nom' vaut 'bruxelles' -->  
<p>{{ nom | upper }}</p>
```

```
<!-- Rendu HTML : <p>BRUXELLES</p> -->
```

- `upper / lower` : Casse du texte.
- `length` : Taille de la collection.
- `default` : Valeur de remplacement si `None`.
- `sort` : Tri alphabétique ou numérique.

Couper une longue description pour un aperçu :

```
<p>{{ feature.description | truncate(50) }}</p>
```

Résultat : "Une superbe fonctionnalité qui permet de..."

Réutilisation de Code

L'Héritage de Templates (extends)

L'héritage est un des concepts permettant de respecter le principe **DRY** (Don't Repeat Yourself) dans les vues.

- **Le Parent** (`base.html`) :
 - Définit le squelette HTML commun (Head, Navbar, Footer).
 - Marque des emplacements dynamiques avec `{% block nom %}`.
- **L'Enfant** (`page.html`) :
 - Doit commencer par `{% extends "base.html" %}`.
 - Remplace ou complète les blocs définis dans le parent.

Note

Si un enfant ne définit pas un bloc, c'est le contenu par défaut du parent qui est affiché.

Le rôle des Blocs (block)

- Un **bloc** est une balise qui définit une section nommée pouvant être "écrasée" par un template enfant.
- **Dans le parent** : Il sert de marqueur de position (*placeholder*) pour indiquer où le contenu spécifique doit être inséré.
- **Dans l'enfant** : Il permet de fournir le contenu spécifique à cette section sans toucher au reste de la structure.
- On peut définir autant de blocs que nécessaire (title, content, scripts, sidebar, etc.).

Exemple d'héritage

Parent (base.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>
    {% block title %}
    {% endblock %}
  </title>
</head>
<body>
  <nav>Menu...</nav>
  {% block content %}
  {% endblock %}
</body>
</html>
```

Enfant (contact.html)

```
{% extends "base.html" %}

{% block title %}Contact{% endblock %}

{% block content %}
  <h1>Contact</h1>
  <p>Email : contact@example.com</p>
{% endblock %}
```

L'Inclusion (include)

Permet d'insérer le contenu d'un autre fichier à l'endroit précis de l'appel. Utile pour segmenter les composants réutilisables.

Template Principal

```
<body>
  {% include "partials/_nav.html" %}

  <main>
    <h1>Bienvenue</h1>
    <p>Contenu de la page...</p>
  </main>
</body>
```

Partiel (_nav.html)

```
<nav>
  <ul>
    <li>
      <a href="/">Accueil</a>
    </li>
    <li>
      <a href="/blog">Blog</a>
    </li>
  </ul>
</nav>
```

Héritage vs Inclusion

L'**héritage** (extends) définit la structure globale de la page (du général au particulier), tandis que l'**inclusion** (include) permet d'insérer des composants réutilisables.

Quand privilégier l'Héritage ?

Évitez de reconstruire le squelette de la page par inclusions successives dans chaque fichier :

À éviter (Inclusion)

```
<!-- profil.html -->
{% include "header.html" %}
{% include "nav.html" %}
<main>
    <h1>Profil Utilisateur</h1>
</main>
{% include "footer.html" %}
```

Répétition manuelle de la structure sur chaque page.

À privilégier (Héritage)

```
<!-- profil.html -->
{% extends "base.html" %}

{% block content %}
<main>
    <h1>Profil Utilisateur</h1>
</main>
{% endblock %}
```

La structure est gérée par le parent, l'enfant ne fournit que le contenu.

Macros

Une **macro** permet de définir des fragments de code réutilisables, agissant comme des fonctions :

- **Factorisation** : Évite la duplication de structures HTML répétitives (boutons, champs de formulaire, cartes).
- **Flexibilité** : Supporte des arguments (positionnels ou nommés) avec des valeurs par défaut.
- **Modularité** : Peut être définie localement ou importée depuis un fichier externe via `{% import %}`.

Exemple de Macro

```
{% macro button(label, color='primary') %}  
  <button class="btn btn-{{ color }}">  
    {{ label }}  
  </button>  
{% endmacro %}
```

```
{{ button('Valider') }}  
{{ button('Annuler', 'danger') }}
```

Sécurité

Auto-escaping et XSS

Jinja2 active par défaut l'**auto-escaping** pour prévenir les failles **XSS** (Cross-Site Scripting) :

- Tout contenu dynamique est traité comme du texte brut par défaut.
- Les caractères réservés (<, >, &, ") sont convertis en entités HTML.

Exemple d'injection filtrée

Variable : nom = "<script>alert('XSS')</script>"

Template : Bonjour {{ nom }}

Rendu HTML :

Bonjour <script>alert('XSS')</script>

Résultat : Le navigateur affiche le texte littéralement au lieu d'exécuter le script malveillant.

Si vous devez VOLONTAIREMENT afficher du HTML :

```
{{ mon_html_propre | safe }}
```

Attention

Ne jamais utiliser `| safe` sur des données saisies par l'utilisateur !

Objets Globaux Flask

Accès automatique aux objets Flask

Flask injecte automatiquement plusieurs objets globaux dans le contexte de rendu de Jinja2 :

- **request** : Contient les informations de la requête HTTP entrante (paramètres d'URL `.args`, données de formulaire `.form`, cookies, méthode).
- **session** : Un dictionnaire permettant de stocker des informations persistantes spécifiques à un utilisateur entre plusieurs requêtes (ex : panier d'achat, état de connexion).
- **g** : Un objet de stockage temporaire dont la durée de vie est limitée à une seule requête. Utile pour partager des données (comme une connexion DB) entre différentes fonctions.
- **config** : Donne accès direct à l'objet de configuration de l'application Flask (`app.config`), permettant d'afficher des paramètres globaux.

Exemples d'utilisation des objets globaux

request : Mettre en gras le lien actif dans le menu

```
<a href="/" class="{% if request.path == '/' %}active{% endif %}">
  Accueil
</a>
```

session : Afficher un message personnalisé si connecté

```
{% if session.user_id %}
  <p>Bienvenue, {{ session.username }} !</p>
{% endif %}
```

config : Accéder aux paramètres de l'application

```
<footer>Contactez le support : {{ config.MAIL_ADMIN }}</footer>
```

Perspective Architeturale

SSR vs CSR vs Hybride

- **Jinja2 (SSR)** : Le serveur génère le HTML final. Simple, robuste et nativement optimisé pour le SEO.
- **React/Vue (CSR)** : Le navigateur construit la page via du Javascript. Offre une grande interactivité mais nécessite une API séparée.
- **Next.js (Hybride)** : Combine le SSR et le CSR (Hydratation). Performance maximale mais complexité accrue (nécessite Node.js).

Pourquoi privilégier Jinja2 avec Flask ?

- **Simplicité** : Pas de "build step" complexe ni de gestion de dépendances NPM.
- **Couplage direct** : Accès immédiat aux données Python sans passer par une API REST.
- **Légèreté** : Temps d'affichage initial ultra-rapide sans exécution de JS lourd.

Questions ?